



UNINASSAU

LISTAS

Análise e Desenvolvimento de Sistemas
Programação Orientada à Objetos e Estruturas de dados
Profº Eng. Esp. Lindenberg Andrade

Interface Collection, Interface List, ArrayList, LinkedList e Vector

INTERFACE COLLECTION

- Collection é a interface raiz de toda a hierarquia de coleções em Java.
 - Define o comportamento básico de qualquer grupo de objetos
 - Não podemos criar objetos de uma interface
 - Usamos como tipo de referência
 - Todas as coleções herdam de Collection

Hierarquia: Collection → List, Set, Queue

Métodos Comuns

`add(element)`

Adiciona um elemento à coleção

`remove(element)`

Remove um elemento da coleção

`size()`

Retorna a quantidade de elementos

`isEmpty()`

Verifica se está vazia

`contains(element)`

Verifica se contém um elemento

`clear()`

Remove todos os elementos

INTERFACE LIST

Ordem

Mantém a ordem de inserção dos elementos

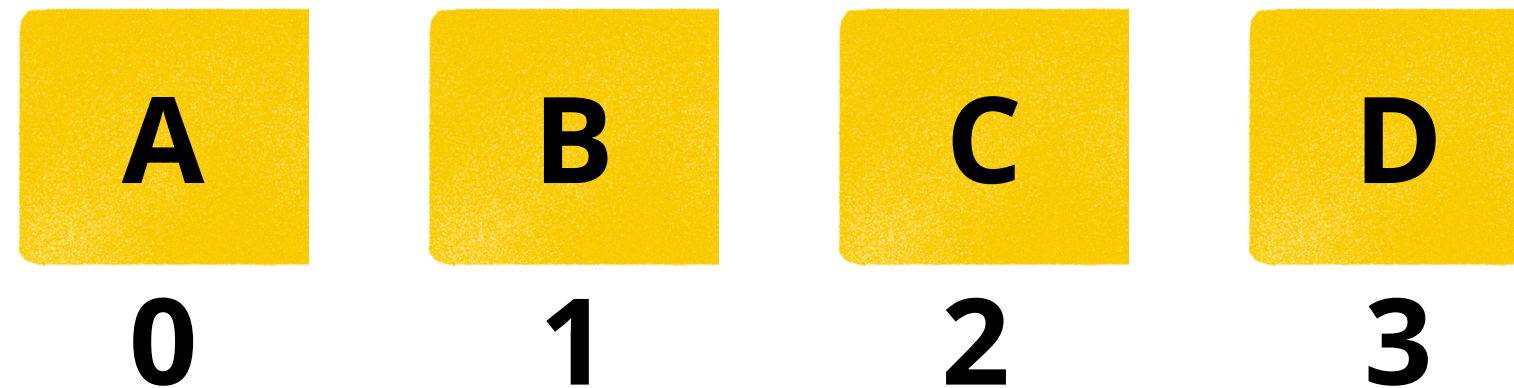
Duplicatas

Permite elementos duplicados

Índice

Acesso por posição (0, 1, 2...)

Representação Visual



- Métodos Específicos

- `get(index)` - Retorna elemento
- `add(index, element)` - Insere em posição
- `remove(index)` - Remove por posição

- Mais Métodos

- `indexOf(element)` - Encontra posição
- `set(index, element)` - Substitui elemento
- `subList(from, to)` - Sublista

ARRAYLIST

Implementação: Usa um Array interno para armazenar os elementos de forma contígua na memória.

Vantagens

- Acesso e busca muito rápidos ($O(1)$)
- Usa menos memória que LinkedList
- Iteração eficiente



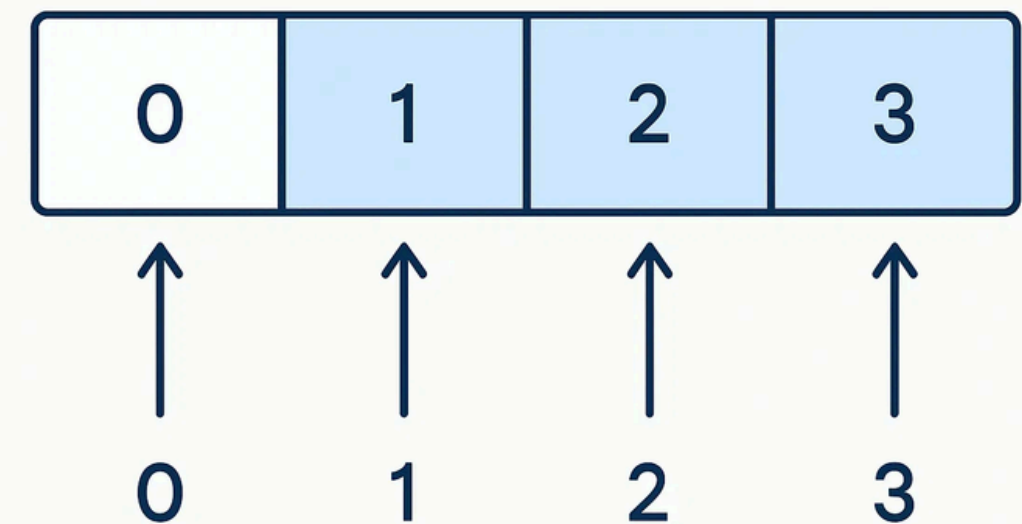
Desvantagens

- Inserção/remoção no meio é lenta ($O(n)$)
- Requer deslocamento de elementos

Uso Ideal:

Quando você precisa acessar elementos com frequência e as alterações são raras.

ArrayList: Array Based



ARRAYLIST EM AÇÃO

Exemplo 1: Criação e Adição

```
1 List<String> frutas = new ArrayList<>();
2 frutas.add("Maçã");
3 frutas.add("Banana");
4 frutas.add("Laranja");
```

RESULTADO:

[Maçã, Banana, Laranja]

Exemplo 2: Acesso e Remoção

```
5 String segunda = frutas.get(1);
6 // Saída: Banana
7 frutas.remove(0);
8 // Remove Maçã
```

RESULTADO FINAL:

[Banana, Laranja]

Fácil de usar mas cuidado com remoções e inserções no início da lista!

LINKEDLIST: LISTA FLEXÍVEL

Implementação: Usa uma estrutura de Nós que se conectam sequencialmente (lista duplamente encadeada).

Vantagens

- Inserção e remoção muito rápidas ($O(1)$)
- Especialmente rápido nas extremidades
- Flexibilidade para modificações frequentes

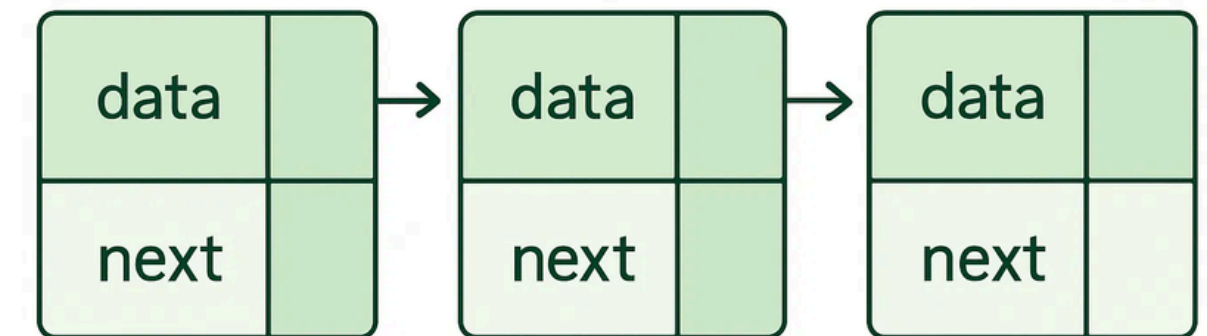


Desvantagens

- Acesso e busca de elementos são lentos ($O(n)$)
- Precisa percorrer a lista do início ou fim
- Usa mais memória (armazena os "links")

Uso Ideal: Quando você precisa adicionar ou remover elementos com frequência, como em filas e pilhas.

LinkedList: Linked List



LINKEDLIST EM AÇÃO

```
LinkedList<Integer> numeros = new LinkedList<>();  
  
numeros.addFirst(10);  
  
numeros.addLast(20);  
  
numeros.addLast(30);  
  
// Remove o primeiro elemento (10)  
  
numeros.removeFirst();  
  
// Acessa o primeiro elemento  
  
System.out.println(numeros.get(0));
```

- Operações Realizadas:
 - addFirst(10): Adiciona 10 no início
 - addLast(20/30): Adiciona 20 e 30 no final
 - removeFirst(): Remove o primeiro elemento

Saída Esperada:
20

Por Quê?

LinkedList é excelente para operações de fila/pilha. Inserção e remoção no início/fim são $O(1)$.

COMPARAÇÃO: ARRAYLIST VS LINKEDLIST

OPERAÇÃO	ARRAYLIST	LINKEDLIST
Acesso (get)	Rápido ($O(1)$)	Lento ($O(n)$)
Inserção/Remoção (Início)	Lenta ($O(n)$)	Rápida ($O(1)$)
Inserção/Remoção (Meio)	Lenta ($O(n)$)	Rápida ($O(1)$)
Inserção/Remoção (Fim)	Rápida ($O(1)$)	Rápida ($O(1)$)
Uso de Memória	Menor	Maior (links)
Iteração	Eficiente	Menos eficiente
Base Interna	Array	Lista Encadeada

Recomendação

Use ArrayList por padrão - é mais rápido para a maioria dos casos. Use LinkedList apenas quando você tiver muitas inserções/remoções no meio ou nas extremidades da lista.

VECTOR: O LEGADO

Implementação

Baseado em Array (como ArrayList)

Sincronização

Sincronizado (thread-safe)

Era

Legado (Java 1.0)

Vector

- Sincronizado (thread-safe)
- Mais lento que ArrayList
- Raramente usado hoje
- Custo de sincronização

ArrayList

- Não sincronizado (mais rápido)
- Melhor performance
- Preferido atualmente
- Sem overhead de sincronização

Por Que Evitar Vector?

Vector é sincronizado, o que o torna mais lento. Se você precisa de sincronização, existem alternativas melhores como `Collections.synchronizedList()` ou `CopyOnWriteArrayList`. Para aplicações single-threaded, ArrayList é sempre a melhor escolha.

Recomendação

Use ArrayList como padrão. Vector é mantido apenas por compatibilidade com código legado. Não use em novos projetos!

LISTAS

CONCEITO DE LISTA

Definição: Formas de organizar e armazenar dados para que possam ser acessados e modificados de forma eficiente.

- Permitem operações rápidas e seguras
- Facilitam o gerenciamento de grandes volumes de dados
- São fundamentais para algoritmos eficientes
- Em Java, as Collections são o principal grupo

Exemplo 1: Lista de Compras

Maçã → Pão → Leite → Ovos

Exemplo 2: Agenda de Contatos

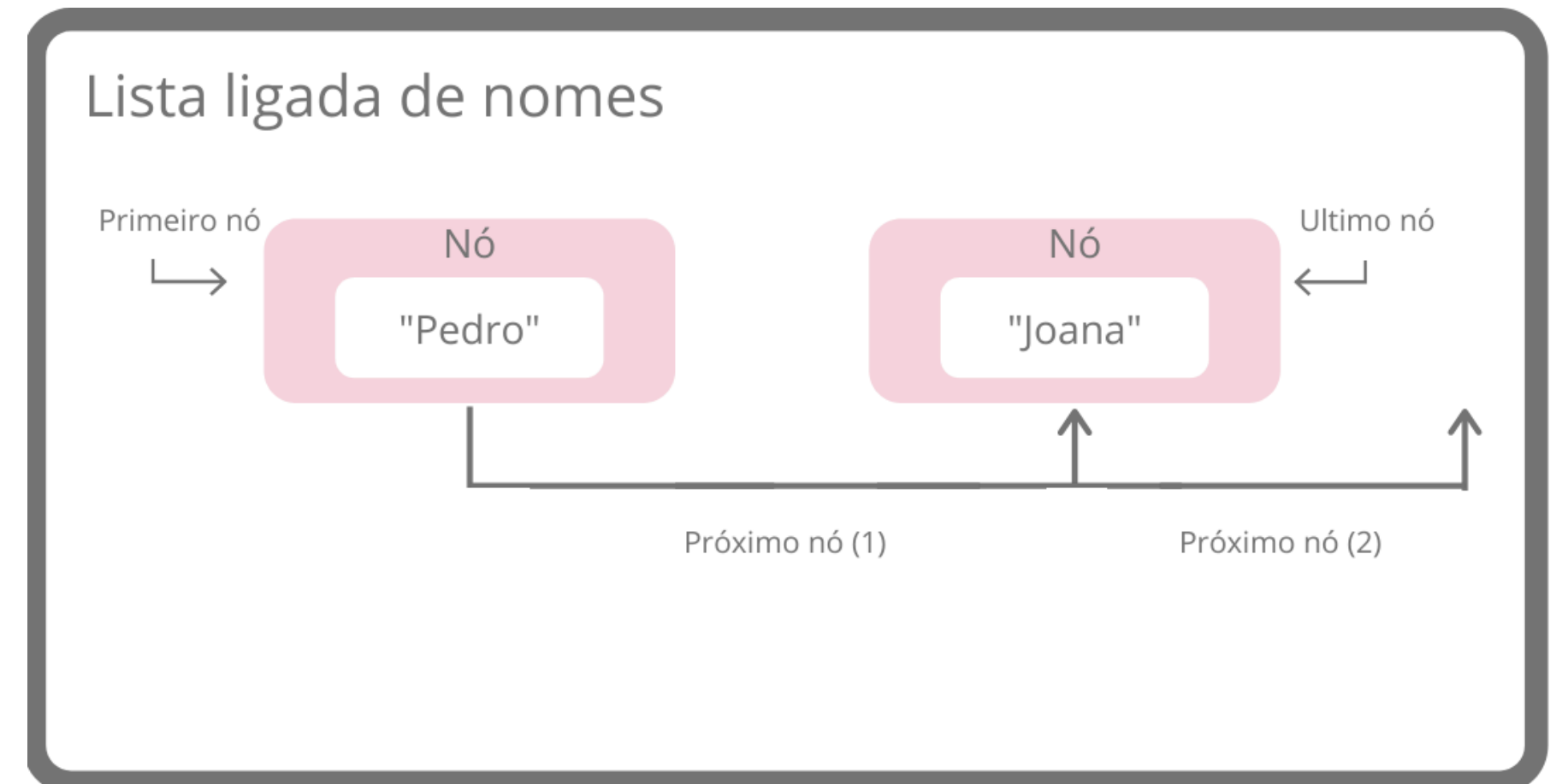
João → Maria → Pedro → Ana

EM JAVA:

java.util.Collections

MAIS SOBRE LISTAS

- Uma lista é uma coleção ordenada de elementos, onde cada item possui uma posição.
- Permite armazenar e acessar dados de forma sequencial.
- Cada elemento é chamado de nó (em listas encadeadas) ou posição (em listas estáticas).
- Principais operações:
 - Inserir elemento
 - Remover elemento
 - Buscar elemento
 - Percorrer elementos



LISTA ESTÁTICA (SEQUENCIAL)

- Armazenada em vetor (array).
- Tamanho fixo definido na criação.
- Acesso direto aos elementos (índices).

Vantagens

- Acesso rápido (índice direto).
- Simples de implementar.



Desvantagens

- Espaço fixo — desperdício ou falta de memória.
- Inserções e remoções custosas (deslocamento).

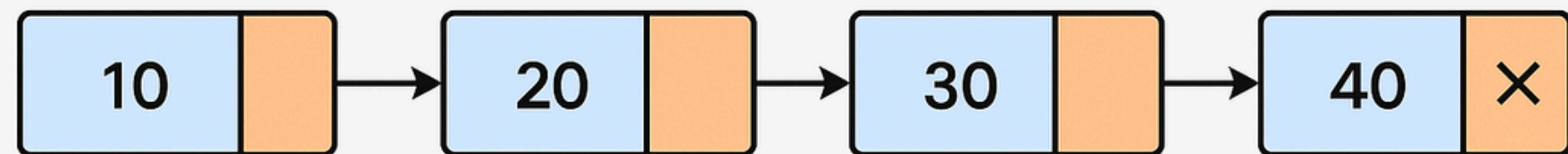
Exemplo
em Java:

```
public class ListaEstatica {  
    private int[] dados;  
    private int tamanho;  
  
    public ListaEstatica(int capacidade) {  
        dados = new int[capacidade];  
        tamanho = 0;  
    }  
  
    public void inserir(int valor) {  
        if (tamanho < dados.length) {  
            dados[tamanho++] = valor;  
        }  
    }  
  
    public void remover(int indice) {  
        for (int i = indice; i < tamanho - 1; i++) {  
            dados[i] = dados[i + 1];  
        }  
        tamanho--;  
    }  
  
    public void exibir() {  
        for (int i = 0; i < tamanho; i++) {  
            System.out.print(dados[i] + " ");  
        }  
        System.out.println();  
    }  
}
```

LISTA ENCADEADA (DINÂMICA)

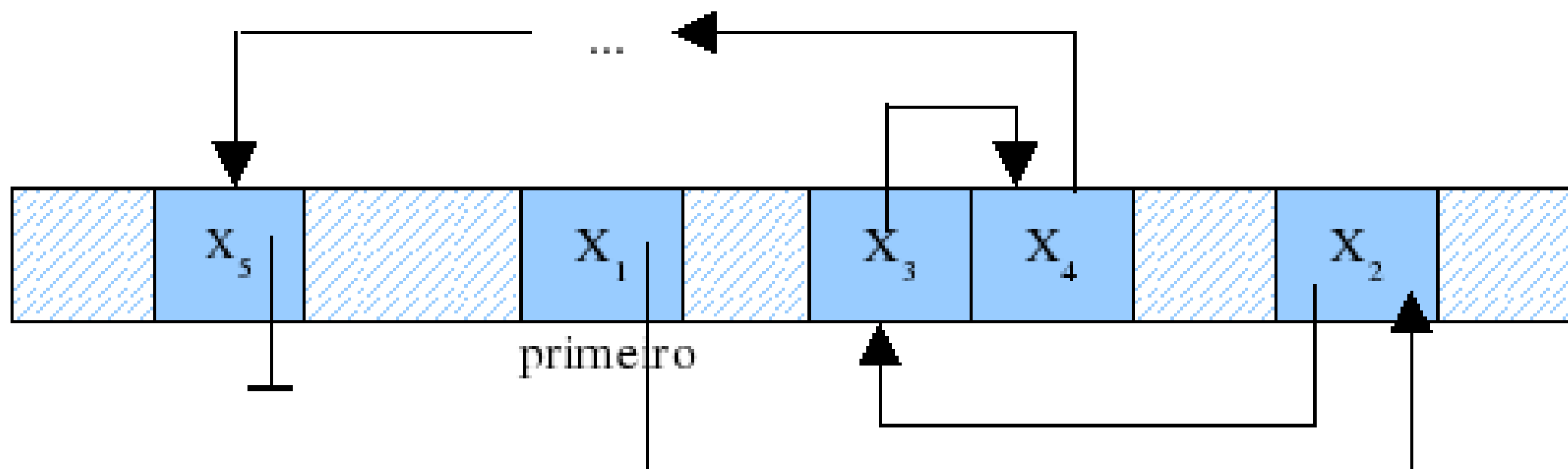
- Cada elemento (nó) contém dados e um ponteiro para o próximo.
- Tamanho variável, cresce ou diminui conforme necessidade.

Lista Encadeada (Dinâmica)



LISTA SIMPLESMENTE ENCADEADA

- Cada nó aponta apenas para o **próximo**.
- Percorrida apenas em uma **direção**.



Numa lista linear simplesmente encadeada cada elemento possui, além do espaço para armazenamento da informação, um espaço para armazenar uma referência da localização na memória onde o próximo elemento da lista (ou o anterior) se encontra.

Vantagens

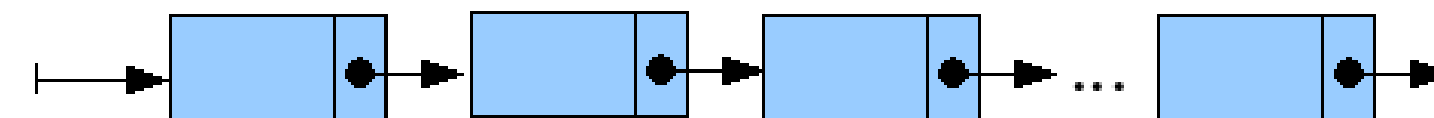
- Tamanho flexível.
- Inserções/remoções fáceis no início.



Desvantagens

- Acesso sequencial (não há índice).
- Mais uso de memória (ponteiros).

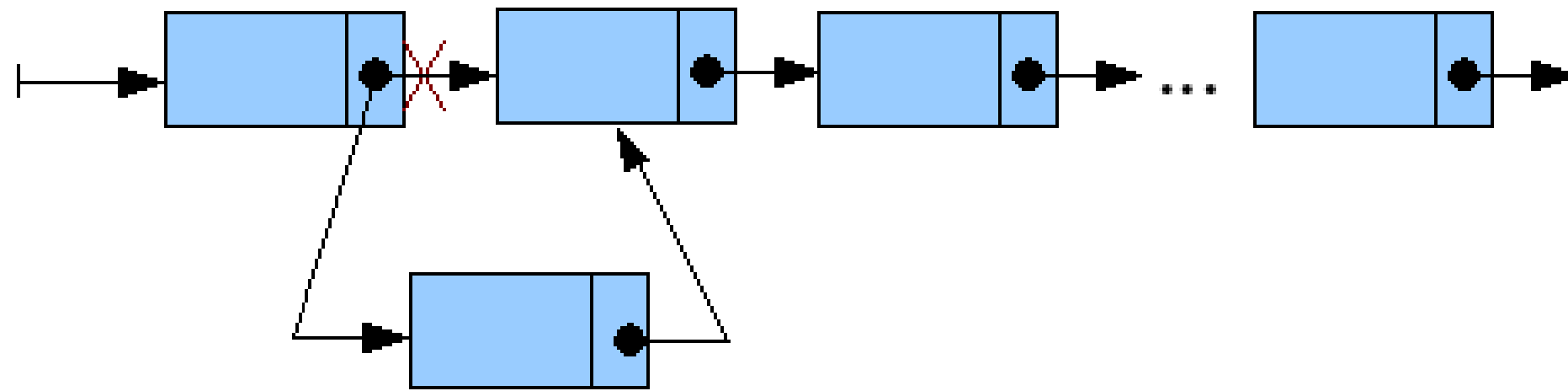
A representação simbólica para a lista linear simplesmente encadeada é apresentada a seguir:



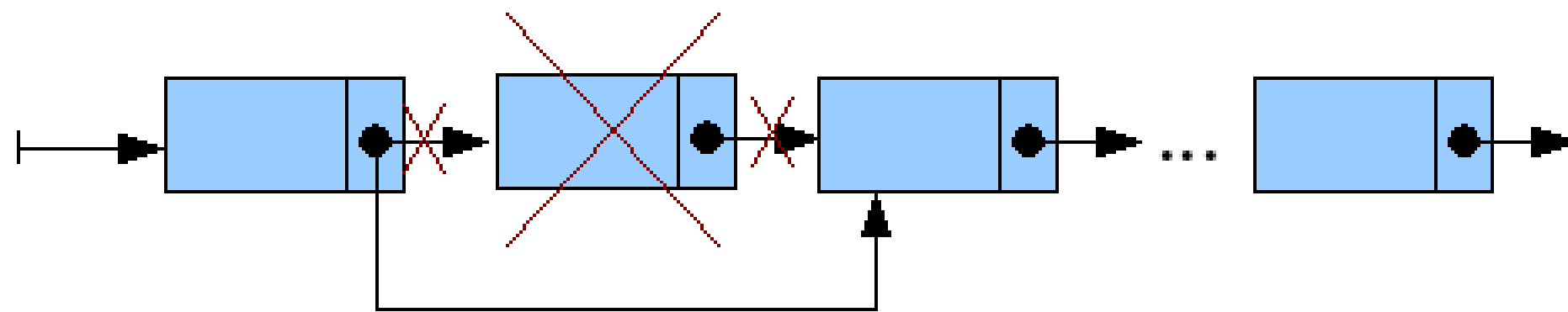
ROTINAS DE MANIPULAÇÃO LISTA SIMPLEMENTE ENCADEADA

- As rotinas inserção e remoção de elementos numa lista são realizadas manipulando-se a referência ao próximo nó da lista. Observe a ilustração a seguir:

Inserção:



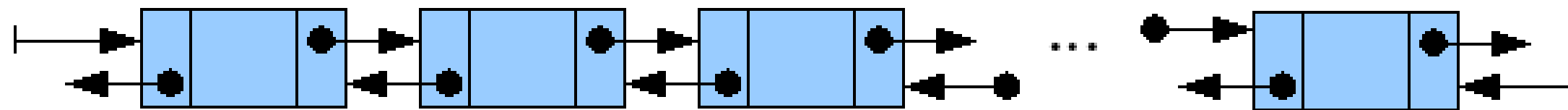
Remoção:



LISTA DUPLAMENTE ENCADEADA

- Cada nó aponta para o **anterior** e o **próximo**.
- Permite percorrer **nos dois sentidos**.

A representação simbólica para a lista linear duplamente encadeada é apresentada a seguir:



Vantagens

- Percurso em ambas as direções.
- Facilita remoções e inserções em qualquer posição.



Desvantagens

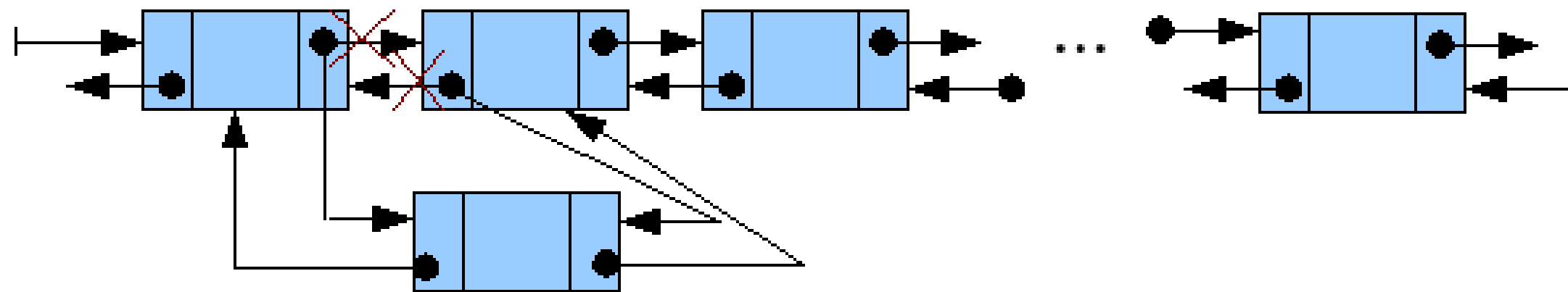
- Estrutura mais complexa.
- Maior consumo de memória (dois ponteiros por nó).

Numa lista linear duplamente encadeada cada elemento possui, além do espaço para armazenamento da informação, um espaço para armazenar a referencia da localização de memória onde se encontra o próximo elemento da lista e outro espaço para armazenar a referencia da localização de memória onde se encontra o elemento anterior.

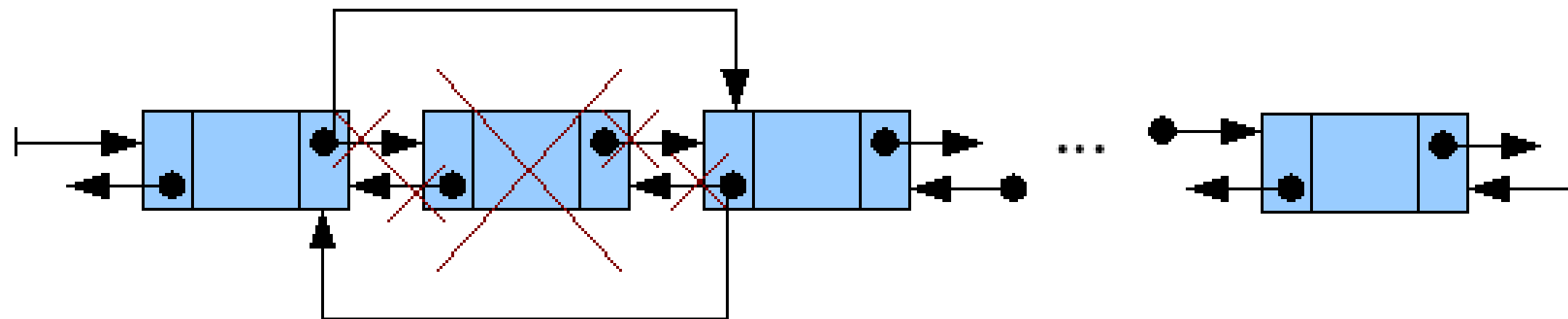
ROTINAS DE MANIPULAÇÃO LISTA DUPLAMENTE ENCADEADA

- As rotinas inserção e remoção de elementos numa lista são realizadas manipulando-se a referência ao próximo nó da lista. Observe a ilustração a seguir:

Inserção:



Remoção:



EXEMPLOS

EXEMPLO 1: LISTA SIMPLESMENTE ENCADEADA

- Cada nó tem apenas o ponteiro para o próximo elemento.

```
ListaSimples.java X
1 package br.com;
2
3 //Lista Simplesmente Encadeada
4 class No {
5     int valor;
6     No prox;
7
8     No(int valor) {
9         this.valor = valor;
10        this.prox = null;
11    }
12 }
13
14 public class ListaSimples {
15     public static void main(String[] args) {
16         // Criação dos nós
17         No n1 = new No(10);
18         No n2 = new No(20);
19         No n3 = new No(30);
20
21         // Encadeamento manual
22         n1.prox = n2;
23         n2.prox = n3;
24
25         // Percorrendo e exibindo
26         No atual = n1;
27         System.out.println("Lista Encadeada Simples:");
28         while (atual != null) {
29             System.out.print(atual.valor + " ");
30             atual = atual.prox;
31         }
32     }
33 }
```

Saída:

Lista Encadeada Simples:
10 20 30

EXEMPLO 2: LISTA DUPLAMENTE ENCADEADA

- Cada nó tem ponteiro para o anterior e o próximo.

```
3 //Lista Duplamente Encadeada
4 class Nodo {
5     int valor;
6     Nodo ant;
7     Nodo prox;
8
9     Nodo(int valor) {
10         this.valor = valor;
11         this.ant = null;
12         this.prox = null;
13     }
14 }
15
```

```
16 public class ListaDuplaSimples {
17     public static void main(String[] args) {
18         Nodo n1 = new Nodo(5);
19         Nodo n2 = new Nodo(15);
20         Nodo n3 = new Nodo(25);
21
22         // Encadeamento duplo
23         n1.prox = n2;
24         n2.ant = n1;
25         n2.prox = n3;
26         n3.ant = n2;
27
28         // Percurso direto
29         System.out.println("Percorrendo do início ao fim:");
30         Nodo atual = n1;
31         while (atual != null) {
32             System.out.print(atual.valor + " ");
33             atual = atual.prox;
34         }
35
36         // Percurso reverso
37         System.out.println("\n\nPercorrendo do fim ao início:");
38         atual = n3;
39         while (atual != null) {
40             System.out.print(atual.valor + " ");
41             atual = atual.ant;
42         }
43     }
44 }
```

Saída:
Percorrendo do início ao fim:
5 15 25

Percorrendo do fim ao início:
25 15 5

Exercícios Propostos

EXERCÍCIO 1: INSERÇÃO NO INÍCIO (LISTA SIMPLEMENTE ENCADEADA)

- Implemente uma classe ListaSimples com os métodos:
- `inserirInicio(int valor)` — insere um novo nó no início da lista.
- `mostrar()` — percorre e exibe todos os valores da lista.
- Desafio:
- Após três inserções (ex: 30, 20, 10), o método `mostrar()` deve exibir:
10, 20, 30

EXERCÍCIO 2: REMOÇÃO DE ELEMENTO (LISTA SIMPLEMENTE ENCADEADA)

- Adapte o código da lista simples para incluir o método:
- `remover(int valor)` — remove o primeiro nó cujo valor seja igual ao informado.

Exemplo de uso:

```
lista.inserirInicio(10);  
lista.inserirInicio(20);  
lista.inserirInicio(30);  
lista.remover(20);
```

Saída esperada:
30 10

EXERCÍCIO 3: INSERÇÃO ORDENADA (LISTA SIMPLES)

- Crie o método `inserirOrdenado(int valor)` que insere o novo elemento na posição correta, mantendo a lista em ordem crescente.

Exemplo de uso:

```
lista.inserirOrdenado(10);  
lista.inserirOrdenado(5);  
lista.inserirOrdenado(20);  
lista.mostrar();
```

Saída esperada:
5 10 20

EXERCÍCIO 4 — LISTA DUPLAMENTE ENCADEADA COM PERCURSO DUPLO

- Crie uma classe ListaDupla com métodos para:
- Inserir elementos no final da lista (inserirFim),
- Exibir a lista do início ao fim (mostrarDireto),
- Exibir a lista do fim ao início (mostrarReverso).

Exemplo de uso:

```
lista.inserirFim(1);  
lista.inserirFim(2);  
lista.inserirFim(3);  
lista.mostrarDireto();  
lista.mostrarReverso();
```

Saída esperada:

Direto: 1 2 3

Reverso: 3 2 1

REFERÊNCIAS

1. BARNES, David J.; KÖLLING, Michael. Programação orientada a objetos com Java: uma introdução prática usando o BlueJ. 4. ed. São Paulo: Pearson Prentice Hall, 2008.
2. SANTOS, Rafael. Introdução à Programação Orientada a Objetos usando Java. 2. ed. Rio de Janeiro: Elsevier, 2013. 336 p.
3. SERSON, Roberto Rubinstein. Programação orientada a objetos com Java 6 – curso universitário. 1. ed. Rio de Janeiro: Brasport, jun. 2009. 492 p.
4. PREISS, Bruno R. Estruturas de dados e algoritmos: padrões de projetos orientados a objetos com Java. Rio de Janeiro: Campus, 2001. 566 p.

Transcendence - A Revolução (2014)



Dr. Will Caster, a maior autoridade do mundo em inteligência artificial, está conduzindo experimentos altamente controversos, na intenção de criar um robô com grande variedade de emoções humanas. Quando extremistas antitecnologia tentam matá-lo, Caster convence sua esposa, Evelyn, e seu melhor amigo, Max Waters, a testar seu novo invento nele mesmo. Só que a grande questão não é se eles podem fazer isto, mas se eles devem dar este passo.

Dúvidas?



Profº Lindenberg Andrade
E-mail: linndemberg1@gmail.com



Additional contacts via QR code